Набор заданий по основам Python

Общие требования

Если в условии не указано иное:

* использовать Python 3;
* основную логику реализовывать в виде функций;
* не использовать глобальные переменные для хранения результатов;
* не использовать сторонние библиотеки, кроме явно указанных в условии;
* для проверки алгоритма самостоятельно подготовить несколько тестовых примеров;
* при работе с файлами корректно обрабатывать пустой файл и некорректные данные;
* если требуется генерация тестовых данных, генератор также должен быть реализован программно.

Вариант 1

Задача 1. Спиральная матрица

Требуется написать функцию, которая принимает два целых числа `n` и `m` и создает матрицу размера `n × m`, заполненную числами от `1` до `n * m` по спирали, начиная с левого верхнего угла и двигаясь по часовой стрелке.

Дополнительные требования:

* функция должна возвращать матрицу, а не печатать ее;
* матрица может быть как квадратной, так и прямоугольной;
* числа должны увеличиваться от `1` к `n * m`;
* запрещается использовать готовые функции или библиотеки для построения спирали;
* дополнительно реализовать режим заполнения от `n * m` до `1`;
* продемонстрировать работу функции для размеров `3 × 3`, `3 × 5` и `4 × 2`.

Задача 2. Самая необычная строка

Требуется написать функцию, которая последовательно получает от пользователя строки до ввода специального слова `STOP` и определяет строку с наибольшим количеством различных символов.

Дополнительные требования:

* все введенные строки нельзя хранить в списке;

* после каждого ввода необходимо при необходимости обновлять текущий результат;
* регистр букв не учитывать;
* пробелы и знаки пунктуации считать символами;
* если несколько строк имеют одинаковый результат, сохранить первую из них;
* после завершения ввода вывести найденную строку и количество различных символов в ней.

Задача 3. Сжатие последовательности

Требуется написать функцию, которая принимает список целых чисел и заменяет каждую группу одинаковых подряд идущих элементов парой `(значение, количество повторений)`.

Например:

```
[1, 1, 1, 2, 2, 5, 5, 5, 5, 3]
```

должна превратиться в:

```
[(1, 3), (2, 2), (5, 4), (3, 1)]
```

Дополнительные требования:

* не использовать `itertools.groupby`;
* порядок групп должен сохраняться;
* отдельно написать функцию обратного преобразования;
* проверить, что после сжатия и последующего восстановления получается исходный список;
* пустой список должен корректно обрабатываться.

Задача 4. Ближайшая точка

Требуется написать функцию, которая принимает список двумерных точек вида `(x, y)` и координаты некоторой контрольной точки `(x0, y0)` и возвращает точку, находящуюся ближе всего к контрольной.

Дополнительные требования:

* использовать евклидово расстояние;
* не использовать сортировку;
* не создавать отдельный список расстояний;
* при одинаковом расстоянии вернуть точку, которая встретилась первой;
* дополнительно вернуть само расстояние до найденной точки;

* объяснить, почему для сравнения расстояний необязательно вычислять квадратный корень.

Задача 5. Скользящее среднее

Требуется написать функцию, которая принимает список чисел и размер окна `k` и возвращает список значений простого скользящего среднего.

Например:

```
[2, 4, 6, 8, 10], k = 3
```

результат:

```
[4, 6, 8]
```

Дополнительные требования:

* среднее для каждого окна вычислять самостоятельно, без `numpy`;
* корректно проверять значение `k`;
* после собственной реализации написать второй вариант с использованием `numpy`;
* сравнить результаты двух реализаций на одном наборе данных;
* объяснить вычислительную сложность первого варианта;
* по возможности реализовать более эффективный вариант, в котором не пересчитывается сумма всего окна с нуля.

Задача 6. Статистика журнала

Требуется написать функцию, которая читает текстовый файл с результатами студентов в формате:

```
Иванов Иван;78
Петров Петр;91
Сидоров Алексей;64
```

и вычисляет:

* средний балл;
* максимальный балл;
* минимальный балл;
* количество студентов, набравших не менее 60 баллов;
* имя студента с максимальным результатом.

Дополнительные требования:

* не загружать весь файл целиком в память;

* обрабатывать файл построчно;
* предусмотреть строки с некорректными данными;
* написать функцию генерации тестового файла;
* для генерации имен использовать `Faker`, а оценки генерировать с помощью `random`;
* генератор должен уметь создавать файл минимум на 1000 записей.

Задача 7. Подсчет слов

Требуется написать функцию, которая принимает текстовый файл и строит словарь, содержащий количество вхождений каждого слова.

Дополнительные требования:

* регистр букв игнорировать;
* знаки `. , ! ? : ;` не должны считаться частью слова;
* вывести 10 наиболее часто встречающихся слов;
* не использовать `collections.Counter`;
* обработку файла выполнять построчно;
* дополнительно создать генератор большого тестового файла на 10 000 предложений;
* текст для генерации должен содержать случайные имена, города и слова, полученные с помощью `Faker`.

Задача 8. Измеритель функции

Требуется написать декоратор `measure_time`, который измеряет время выполнения любой функции и выводит результат.

Например:

```
@measure_time
def calculate():
    ...
```

Дополнительные требования:

* декоратор должен работать с функциями с любым количеством позиционных и именованных аргументов;
* декоратор не должен изменять результат исходной функции;
* использовать `functools.wraps`;
* время измерять с помощью `time.perf_counter`;
* продемонстрировать работу декоратора на трех функциях;

* одна из функций должна принимать аргументы и возвращать значение.

Задача 9. Банковский счет

Требуется реализовать систему классов для моделирования банковского счета.

Необходимо создать класс `BankAccount`, который хранит:

* владельца счета;
* текущий баланс;
* историю операций.

Класс должен предоставлять методы:

```
deposit()
withdraw()
get_balance()
get_history()
```

Дополнительные требования:

* нельзя напрямую изменять баланс извне класса;
* нельзя снять больше денег, чем есть на счете;
* нельзя внести отрицательную сумму;
* каждая операция должна попадать в историю;
* реализовать два класса-наследника: `SavingsAccount` и `CreditAccount`;
* правила снятия денег у классов-наследников должны отличаться;
* продемонстрировать полиморфизм при работе со списком разных счетов.

Задача 10. Система обработки команд

Требуется реализовать систему классов для обработки текстовых команд пользователя.

Пользователь может вводить команды:

```
add 15
multiply 3
subtract 4
divide 2
history
exit
```

Необходимо создать базовый класс команды и отдельные классы для команд `add`, `multiply`, `subtract` и `divide`.

Дополнительные требования:

* каждая команда должна быть отдельным объектом;
* создать класс `CommandProcessor`, отвечающий за выполнение команд;
* результат каждой операции сохранять в истории;
* реализовать обработку некорректных команд;
* деление на ноль должно обрабатываться без аварийного завершения программы;
* использовать наследование и полиморфизм;
* добавить декоратор, который логирует выполнение каждой команды;
* декоратор не должен знать внутреннее устройство конкретных команд.

Вариант 2

Задача 1. Диагональное заполнение

Требуется написать функцию, которая принимает размеры матрицы `n × m` и заполняет ее числами от `1` до `n * m` по диагоналям.

Например, для матрицы `3 × 3` заполнение должно иметь вид:

```
1 2 4
3 5 7
6 8 9
```

Дополнительные требования:

* поддержать прямоугольные матрицы;
* функция должна возвращать матрицу;
* направление заполнения должно задаваться дополнительным параметром;
* не использовать `numpy`;
* отдельно проверить матрицы `2 × 5` и `5 × 2`.

Задача 2. Анализ последовательного ввода

Требуется написать функцию, которая получает от пользователя целые числа до ввода `0` и определяет:

* максимальное число;
* минимальное число;
* количество четных чисел;
* длину самой длинной последовательности положительных чисел, идущих подряд.

Дополнительные требования:

* все числа нельзя сохранять в список;
* информация должна обновляться после каждого ввода;
* само число `0` не включается в анализ;
* если пользователь сразу ввел `0`, программа должна вывести понятный результат;
* запрещается использовать `max()`, `min()` и сортировку.

Задача 3. Палиндром с учетом структуры

Требуется написать функцию, которая определяет, является ли строка палиндромом после удаления пробелов и знаков пунктуации и без учета регистра.

Например:

```
"Аргентина манит негра"
```

должна считаться палиндромом.

Дополнительные требования:

* не использовать `reversed()` и `[::-1]`;
* реализовать проверку с помощью двух индексов;
* учитывать только буквы и цифры;
* вернуть `True` или `False`;
* отдельно вывести очищенную строку и результат проверки.

Задача 4. Пересечение интервалов

Требуется написать функцию, которая принимает два списка числовых интервалов:

```
[(1, 5), (10, 15), (20, 25)]
```

и

```
[(3, 8), (12, 18), (22, 30)]
```

и возвращает интервалы их пересечения.

Для приведенного примера результат:

```
[(3, 5), (12, 15), (22, 25)]
```

Дополнительные требования:

* интервалы считать включающими границы;
* корректно обрабатывать отсутствие пересечений;

* не использовать сторонние библиотеки;
* предусмотреть случай, когда интервалы внутри каждого списка пересекаются между собой;
* не использовать готовые библиотеки для работы с интервалами.

Задача 5. Дисперсия и стандартное отклонение

Требуется написать функцию, которая принимает список чисел и вычисляет:

* среднее арифметическое;
* дисперсию;
* стандартное отклонение.

Дополнительные требования:

* формулы реализовать самостоятельно;
* не использовать `numpy` на первом этапе;
* отдельно реализовать выборочную и генеральную дисперсию;
* затем реализовать аналогичное вычисление с помощью `numpy`;
* сравнить результаты;
* объяснить различие деления на `n` и `n - 1`.

Задача 6. Анализ температуры

Требуется написать функцию, которая обрабатывает CSV-файл с данными:

```
date,temperature
2026-01-01,3.5
2026-01-02,4.1
...
```

и определяет:

* среднюю температуру;
* самый теплый день;
* самый холодный день;
* количество дней с температурой выше средней;
* максимальный перепад температуры между соседними днями.

Дополнительные требования:

* файл обрабатывать построчно;

* не использовать `pandas`;
* некорректные строки пропускать;
* написать генератор CSV-файла на 365 дней;
* даты генерировать автоматически;
* температуры генерировать с использованием `random`;
* дополнительно проверить программу на специально созданном маленьком файле.

Задача 7. Поиск повторяющихся записей

Требуется написать функцию, которая читает текстовый файл со списком электронных адресов, по одному адресу на строку, и определяет:

* количество уникальных адресов;
* количество повторных записей;
* наиболее часто встречающийся адрес.

Дополнительные требования:

* регистр символов в адресе не учитывать;
* пустые строки игнорировать;
* использовать `set` и `dict`;
* не использовать `Counter`;
* создать генератор файла минимум на 5000 адресов;
* адреса генерировать с помощью `Faker`;
* часть адресов должна специально дублироваться.

Задача 8. Кэширование результатов

Требуется написать декоратор `cache`, который запоминает результаты вызова функции и при повторном вызове с теми же аргументами возвращает сохраненный результат без повторного выполнения функции.

Дополнительные требования:

* поддержать позиционные аргументы;
* поддержать именованные аргументы;
* использовать словарь для хранения кэша;
* показать работу на функции, которая искусственно выполняется несколько секунд;

* вывести количество фактических вычислений;
* предусмотреть возможность ограничения размера кэша;
* объяснить, какие аргументы функции можно использовать в качестве ключей словаря.

Задача 9. Геометрические фигуры

Требуется реализовать систему классов для работы с геометрическими фигурами.

Необходимо создать абстрактный базовый класс `Shape` с методами:

```
area()
perimeter()
```

и классы:

```
Rectangle
Circle
Triangle
```

Дополнительные требования:

* каждый класс должен реализовать собственный расчет площади и периметра;
* параметры фигуры должны проверяться при создании;
* создать список объектов разных типов;
* вычислить суммарную площадь всех фигур;
* использовать полиморфизм;
* нельзя проверять тип объекта через большое количество `if isinstance(...)`;
* добавить метод для получения текстового описания фигуры.

Задача 10. Система обработки файлов

Требуется реализовать систему классов, которые обрабатывают файлы разных типов.

Необходимо создать базовый класс `FileProcessor` и классы:

```
TextFileProcessor
CSVFileProcessor
```

Базовый класс должен определять общий интерфейс обработки файла.

Дополнительные требования:

* `TextFileProcessor` должен считать количество строк и слов;
* `CSVFileProcessor` должен считать количество записей и столбцов;
* создать функцию, которая получает объект процессора и запускает обработку;

* использовать полиморфизм;
* ошибки чтения файла обрабатывать внутри системы;
* добавить декоратор журналирования, который записывает имя файла и результат операции;
* не дублировать код открытия файла в каждом классе без необходимости.

Вариант 3

Задача 1. Матрица с шахматным узором

Требуется написать функцию, которая принимает размеры `n × m` и создает матрицу, заполненную символами `0` и `1` в шахматном порядке.

Например:

```
0 1 0 1 0
1 0 1 0 1
0 1 0 1 0
```

Дополнительные требования:

* первый элемент задается параметром функции;
* использовать только базовые операции Python;
* не использовать готовые матричные библиотеки;
* реализовать второй режим, при котором вместо `0` и `1` используются два произвольных символа;
* проверить работу на нескольких прямоугольных матрицах.

Задача 2. Самая длинная серия

Требуется написать функцию, которая принимает последовательность целых чисел и определяет длину самой длинной серии одинаковых элементов, идущих подряд.

Например:

```
[1, 1, 1, 2, 2, 3, 3, 3, 3]
```

результат:

```
4
```

Дополнительные требования:

* не использовать `itertools`;
* не использовать `max`;

* не хранить отдельные серии;
* дополнительно вернуть значение элемента, который образует самую длинную серию;
* при одинаковой длине выбрать первую серию.

Задача 3. Частотный словарь символов

Требуется написать функцию, которая принимает строку и определяет три наиболее часто встречающихся символа.

Дополнительные требования:

* пробелы игнорировать;
* регистр не учитывать;
* при одинаковой частоте первым должен идти символ, который встретился раньше;
* не использовать `Counter`;
* не использовать сортировку всей строки или списка символов;
* функция должна возвращать структуру данных, которую можно использовать дальше в программе.

Задача 4. Разворот матрицы

Требуется написать функцию, которая принимает двумерный список и возвращает новую матрицу, полученную поворотом исходной на 90 градусов по часовой стрелке.

Например:

```
1 2 3
4 5 6
```

превращается в:

```
4 1
5 2
6 3
```

Дополнительные требования:

* поддержать прямоугольные матрицы;
* не использовать `numpy`;
* исходную матрицу не изменять;
* отдельно реализовать поворот на 90 градусов против часовой стрелки;
* проверить результат на нескольких размерах.

Задача 5. Поиск аномалий

Требуется написать функцию, которая принимает список чисел и определяет элементы, отклоняющиеся от среднего значения более чем на `k` стандартных отклонений.

Дополнительные требования:

* среднее и стандартное отклонение реализовать самостоятельно;
* не использовать `numpy` на первом этапе;
* `k` передавать параметром;
* вернуть список найденных аномальных значений;
* затем реализовать аналогичный вариант с `numpy`;
* сравнить результаты двух реализаций.

Задача 6. Анализ продаж

Требуется написать программу, которая читает CSV-файл:

```
date,product,quantity,price
2026-01-01,Book,3,500
2026-01-01,Pen,10,80
...
```

и вычисляет:

* общую сумму продаж;
* самый продаваемый товар по количеству;
* товар с максимальной выручкой;
* среднюю стоимость одной покупки;
* день с максимальной выручкой.

Дополнительные требования:

* файл обрабатывать построчно;
* не использовать `pandas`;
* некорректные записи игнорировать;
* написать генератор файла минимум на 5000 продаж;
* названия товаров выбирать из заранее заданного списка;
* даты и количества генерировать через `random`;
* часть записей должна содержать ошибки для проверки обработки исключений.

Задача 7. Поиск наиболее похожих строк

Требуется написать функцию, которая получает список строк и находит две строки с наименьшим количеством различий по позициям.

Например:

```
"abcdef"
"abcxef"
```

имеют одно различие.

Дополнительные требования:

* сравнивать только строки одинаковой длины;
* расстояние между строками вычислять самостоятельно;
* не использовать библиотеку для вычисления расстояния;
* не использовать сортировку всех возможных пар;
* вернуть сами строки и количество различий;
* отдельно обработать ситуацию, когда подходящей пары нет.

Задача 8. Декоратор повторного запуска

Требуется написать декоратор `retry`, который автоматически повторяет вызов функции, если функция завершилась исключением.

Например:

```
@retry(3)
def load_data():
    ...
```

Дополнительные требования:

* число попыток задается при создании декоратора;
* если функция успешно завершилась, повторные вызовы прекращаются;
* после исчерпания попыток последнее исключение должно быть передано вызывающему коду;
* использовать декоратор с параметром;
* сохранить исходные метаданные функции через `functools.wraps`;
* продемонстрировать работу на функции, которая случайным образом завершается ошибкой.

Задача 9. Товары интернет-магазина

Требуется реализовать систему классов для представления товаров интернет-магазина.

Необходимо создать базовый класс `Product` с:

* названием;
* ценой;
* количеством на складе.

Создать наследников:

```
Book
Electronic
Food
```

Дополнительные требования:

* каждый тип товара должен иметь дополнительное свойство;
* реализовать метод вычисления конечной цены;
* для электронных товаров предусмотреть гарантию;
* для продуктов предусмотреть срок годности;
* для книг предусмотреть автора;
* создать класс `Store`, который хранит товары;
* реализовать поиск товара по названию;
* реализовать продажу товара;
* нельзя продавать товара больше, чем есть на складе;
* использовать наследование и полиморфизм.

Задача 10. Планировщик задач

Требуется реализовать систему классов для управления задачами пользователя.

Необходимо создать класс `Task` с:

* названием;
* приоритетом;
* статусом;
* сроком выполнения.

Необходимо создать класс `TaskManager`.

Дополнительные требования:

* реализовать добавление и удаление задач;
* реализовать изменение статуса;
* реализовать получение задач с высоким приоритетом;
* реализовать поиск просроченных задач;
* реализовать сохранение задач в JSON-файл;
* реализовать загрузку задач из JSON-файла;
* использовать `dataclass` хотя бы для одного класса;
* добавить декоратор, который регистрирует время выполнения операций `TaskManager`;
* предусмотреть обработку поврежденного JSON-файла.

Вариант 4

Задача 1. Треугольник Паскаля

Требуется написать функцию, которая принимает число `n` и строит первые `n` строк треугольника Паскаля.

Например:

```
1
1 1
1 2 1
1 3 3 1
```

Дополнительные требования:

* использовать списки;
* не использовать библиотеку `math`;
* каждая следующая строка должна строиться на основе предыдущей;
* отдельно реализовать функцию, возвращающую только строку с номером `k`;
* корректно обрабатывать `n = 0`.

Задача 2. Числа с особыми свойствами

Требуется написать функцию, которая принимает два целых числа `a` и `b` и возвращает все числа из диапазона `[a, b]`, у которых сумма цифр равна произведению ненулевых цифр.

Например, для числа `123`:

```
1 + 2 + 3 = 6
1 * 2 * 3 = 6
```

поэтому число подходит.

Дополнительные требования:

* сумму и произведение цифр вычислять самостоятельно;
* не преобразовывать число в строку;
* корректно обрабатывать число `0`;
* отрицательные числа считать недопустимыми;
* дополнительно посчитать количество найденных чисел.

Задача 3. Удаление дубликатов с сохранением порядка

Требуется написать функцию, которая принимает список произвольных хешируемых объектов и возвращает список без повторений, сохраняя порядок первого появления элементов.

Например:

```
[3, 1, 3, 2, 1, 5, 2]
```

должен превратиться в:

```
[3, 1, 2, 5]
```

Дополнительные требования:

* не использовать `set` как единственный механизм решения;
* порядок должен сохраняться;
* сложность решения должна быть близка к `O(n)`;
* объяснить, зачем в решении можно использовать `set`;
* отдельно проверить работу со строками.

Задача 4. Ходы шахматного коня

Требуется написать функцию, которая получает положение коня на шахматной доске `8 × 8` и возвращает все клетки, на которые он может перейти за один ход.

Дополнительные требования:

* координаты клетки передавать в виде `(row, column)`;
* не использовать библиотеку для работы с шахматами;
* не возвращать координаты, выходящие за пределы доски;
* результат отсортировать по номеру строки и столбца;

* дополнительно написать функцию, которая проверяет, может ли конь перейти с одной заданной клетки на другую.

Задача 5. Линейная регрессия

Требуется написать функцию, которая по двум спискам чисел `x` и `y` вычисляет коэффициенты линейной зависимости:

```
y = a * x + b
```

с использованием метода наименьших квадратов.

Дополнительные требования:

* формулы реализовать самостоятельно;
* не использовать `numpy` на первом этапе;
* затем реализовать вычисление с использованием `numpy`;
* сравнить коэффициенты;
* написать функцию, которая по найденным `a` и `b` предсказывает значение `y` для нового `x`;
* корректно обработать случай одинаковых значений `x`.

Задача 6. Лог веб-сервера

Требуется написать функцию, которая анализирует текстовый файл с логами веб-сервера в формате:

```
2026-09-01 12:15:31 192.168.1.10 GET /index.html 200
2026-09-01 12:15:32 192.168.1.11 GET /login 404
```

и определяет:

* количество запросов;
* количество уникальных IP-адресов;
* наиболее активный IP;
* количество запросов каждого HTTP-метода;
* количество ошибок с кодами `4xx`;
* наиболее часто запрашиваемый URL.

Дополнительные требования:

* файл обрабатывать построчно;
* не использовать `pandas`;
* некорректные строки пропускать;

* написать генератор тестового лога минимум на 10 000 строк;
* IP-адреса генерировать с использованием `random`;
* URL и другие данные генерировать случайным образом;
* часть строк должна быть специально сделана некорректной.

Задача 7. Слияние интервалов

Требуется написать функцию, которая принимает список интервалов и объединяет пересекающиеся интервалы.

Например:

```
[(1, 4), (3, 8), (10, 12), (11, 15)]
```

должно превратиться в:

```
[(1, 8), (10, 15)]
```

Дополнительные требования:

* интервалы считать включающими границы;
* реализовать решение самостоятельно;
* разрешается использовать сортировку;
* отдельно обработать вложенные интервалы;
* пустой список должен возвращать пустой список;
* объяснить, почему предварительная сортировка существенно упрощает алгоритм.

Задача 8. Декоратор проверки типов

Требуется написать параметризованный декоратор `validate_types`, который проверяет типы аргументов функции перед ее выполнением.

Пример использования:

```
@validate_types(int, int)
def add(a, b):
    return a + b
```

Дополнительные требования:

* количество переданных типов должно соответствовать количеству обязательных позиционных аргументов;
* при неправильном типе должно возникать понятное исключение `TypeError`;
* декоратор должен работать с именованными аргументами;

* сохранить `__name__` и `__doc__` исходной функции;
* поддержать функции с произвольным количеством аргументов;
* отдельно продемонстрировать работу декоратора на трех функциях.

Задача 9. Система пользователей

Требуется реализовать систему классов для управления пользователями приложения.

Необходимо создать базовый класс `User` и классы:

```
Admin
Student
Teacher
```

Дополнительные требования:

* у каждого пользователя должны быть имя и уникальный идентификатор;
* `Admin` может создавать и удалять пользователей;
* `Teacher` может создавать учебные задания;
* `Student` может решать задания;
* реализовать проверку прав доступа;
* создать класс `Application`, управляющий пользователями;
* использовать полиморфизм;
* нельзя хранить права доступа исключительно в виде большого количества `if` по типу класса;
* добавить сериализацию пользователей в JSON.

Задача 10. Система оценки работ

Требуется реализовать систему классов для автоматической проверки учебных работ.

Необходимо создать:

```
Student
Assignment
Submission
Grader
```

Дополнительные требования:

* `Student` хранит информацию о студенте;
* `Assignment` хранит условие и максимальный балл;
* `Submission` хранит решение студента;

* `Grader` проверяет работу;
* реализовать минимум два типа проверки: числовую и текстовую;
* разные типы проверок должны быть реализованы разными классами;
* `Grader` должен работать с ними через общий интерфейс;
* результаты проверки сохранять в JSON;
* добавить декоратор, который измеряет время проверки;
* реализовать отчет по студенту: количество работ, средний балл, максимальный и минимальный результат.

Вариант 5

Задача 1. Матрица по диагоналям

Требуется написать функцию, которая принимает квадратный размер `n` и заполняет матрицу числами от `1` до `n²` по концентрическим слоям, но направление движения внутри каждого следующего слоя должно чередоваться.

Например, внешний слой заполняется по часовой стрелке, следующий — против часовой стрелки.

Дополнительные требования:

* поддержать как минимум `n = 3`, `n = 4`, `n = 5`;
* функция должна возвращать матрицу;
* не использовать сторонние библиотеки;
* не использовать готовые алгоритмы построения матриц;
* отдельно продемонстрировать результат для нечетного и четного `n`.

Задача 2. Последовательный анализ строк

Требуется написать функцию, которая последовательно получает от пользователя строки до ввода `END` и определяет строку, в которой отношение количества гласных букв к общей длине строки максимально.

Дополнительные требования:

* все строки нельзя сохранять;
* регистр букв не учитывать;
* пробелы не учитывать при вычислении отношения;
* при одинаковом отношении выбрать первую строку;
* пустые строки не должны приводить к делению на ноль;

* после завершения вывести найденную строку и вычисленное отношение.

Задача 3. Разбиение списка

Требуется написать функцию, которая принимает список целых чисел и разделяет его на максимальное количество непрерывных групп так, чтобы внутри каждой группы числа шли в строго возрастающем порядке.

Например:

```
[1, 3, 5, 2, 4, 7, 1, 8]
```

должен быть разбит на:

```
[1, 3, 5]
[2, 4, 7]
[1, 8]
```

Дополнительные требования:

* равные соседние элементы должны начинать новую группу;
* функция должна возвращать список групп;
* дополнительно вернуть длину самой длинной группы;
* не использовать сортировку;
* не изменять исходный список.

Задача 4. Лабиринт

Требуется написать функцию, которая принимает двумерный список символов, представляющий небольшой лабиринт:

```
# # # # # #
# S . . . #
# # # . # #
# . . . . #
# . # # E #
# # # # # #
```

где:

```
# – стена
. – свободная клетка
S – начало
E – конец
```

Функция должна определить, существует ли путь от `S` до `E`.

Дополнительные требования:

* перемещение разрешено только вверх, вниз, влево и вправо;

* реализовать поиск самостоятельно;
* использовать BFS или DFS;
* дополнительно вернуть длину найденного пути;
* если пути нет, вернуть соответствующий результат;
* не использовать готовые библиотеки поиска пути.

Задача 5. Медиана и квантили

Требуется написать функции, которые вычисляют для набора чисел:

* медиану;
* первый квартиль;
* третий квартиль.

Дополнительные требования:

* алгоритм вычисления реализовать самостоятельно;
* разрешается использовать сортировку;
* явно определить, как обрабатывается четное количество элементов;
* не использовать `numpy` на первом этапе;
* затем реализовать аналогичный расчет с помощью `numpy`;
* сравнить результаты;
* проверить работу на наборах из четного и нечетного количества элементов.

Задача 6. Анализ финансовых операций

Требуется написать функцию, которая читает CSV-файл:

```
date,category,amount,type
2026-01-01,food,1500,expense
2026-01-02,salary,100000,income
```

и определяет:

* общую сумму доходов;
* общую сумму расходов;
* разницу между доходами и расходами;
* категорию с наибольшими расходами;
* самый большой отдельный расход;

* месяц с наибольшими расходами.

Дополнительные требования:

* файл обрабатывать построчно;
* не использовать `pandas`;
* некорректные строки пропускать;
* создать генератор тестового файла минимум на 5000 операций;
* категории генерировать из заданного списка;
* суммы создавать через `random`;
* даты генерировать автоматически;
* соотношение доходов и расходов сделать реалистичным: доходов должно быть существенно меньше по количеству, но они могут быть значительно больше по сумме.

Задача 7. Анализ текста

Требуется написать функцию, которая анализирует текстовый файл и определяет:

* количество предложений;
* среднюю длину предложения в словах;
* самое длинное предложение;
* слово, встречающееся чаще всего;
* долю уникальных слов от общего количества слов.

Дополнительные требования:

* считать предложениями фрагменты, заканчивающиеся `.`, `!` или `?`;
* регистр слов игнорировать;
* знаки пунктуации не учитывать;
* файл читать построчно;
* не использовать `nltk`, `spacy` и аналогичные библиотеки;
* создать генератор тестового текста минимум на 5000 предложений;
* для генерации использовать `Faker`;
* генератор должен создавать как короткие, так и длинные предложения.

Задача 8. Декоратор ограничения вызовов

Требуется написать декоратор `limit_calls`, который разрешает функции выполниться только определенное количество раз.

Пример:

```
@limit_calls(3)
def hello():
    print("Hello")
```

После трех успешных вызовов дальнейшие попытки должны приводить к исключению.

Дополнительные требования:

* количество допустимых вызовов задается параметром декоратора;
* декоратор должен работать с функциями, имеющими аргументы;
* результат функции должен возвращаться без изменений;
* сохранить метаданные исходной функции;
* счетчик вызовов должен быть независимым для каждой декорированной функции;
* дополнительно реализовать вариант, в котором вместо исключения функция возвращает специальное значение.

Задача 9. Система доставки

Требуется реализовать систему классов для расчета стоимости доставки.

Необходимо создать базовый класс `Delivery` и классы:

```
CourierDelivery
PostDelivery
PickupDelivery
```

Каждый способ доставки должен рассчитывать стоимость по собственным правилам.

Дополнительные требования:

* стоимость должна зависеть от веса посылки;
* курьерская доставка дополнительно зависит от расстояния;
* почтовая доставка должна иметь фиксированную базовую стоимость;
* самовывоз должен быть бесплатным;
* параметры посылки должны проверяться;
* создать класс `Order`, содержащий товары и способ доставки;
* реализовать расчет полной стоимости заказа;
* использовать полиморфизм;

* пользовательский код не должен проверять конкретный тип доставки через цепочку `if/elif`.

Задача 10. Система обработки данных

Требуется реализовать систему классов для последовательной обработки набора числовых данных.

Необходимо создать классы:

```
DataSource
DataFilter
DataAnalyzer
ReportGenerator
```

Система должна выполнять следующий процесс:

```
получение данных
      ↓
фильтрация
      ↓
анализ
      ↓
формирование отчета
```

Дополнительные требования:

* `DataSource` должен уметь получать данные из списка и из текстового файла;
* `DataFilter` должен уметь удалять отрицательные значения и значения, выходящие за заданный диапазон;
* `DataAnalyzer` должен вычислять минимум, максимум, среднее и стандартное отклонение;
* `ReportGenerator` должен формировать текстовый отчет;
* классы должны взаимодействовать через понятные интерфейсы;
* реализовать как минимум два разных источника данных;
* реализовать как минимум два фильтра;
* анализатор не должен знать, откуда были получены данные;
* добавить декоратор измерения времени для этапов обработки;
* результаты отчета сохранить в отдельный файл;
* для проверки создать генератор входного файла минимум на 10 000 чисел с использованием `random`;
* отдельно проверить работу системы на маленьком заранее известном наборе данных.